

# Análisis Numérico II / Álgebra Lineal Numérica

## Clase 07: Descomposición QR



Facultad de Matemática,  
Astronomía, Física y  
Computación

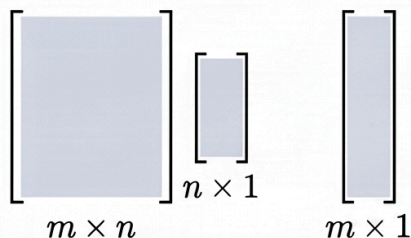


Universidad  
Nacional  
de Córdoba

# Solución de Cuadrados Mínimos: De la Elegancia Matemática a la Trampa Numérica

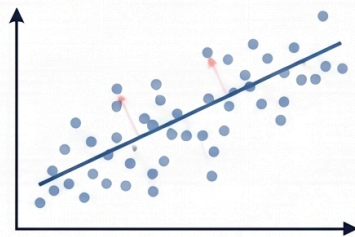
## 1. El Planteo: Sistemas Sobredeterminados

- Sistemas con  $m > n$



En la práctica científica, a menudo tenemos más ecuaciones ( $m$ ) que incógnitas ( $n$ ), lo que genera un sistema inconsistente donde  $Ax = b$  no tiene solución exacta.

- Minimización del Residuo



En lugar de buscar la igualdad, buscamos el vector  $x$  que minimice la norma Euclídea del residuo:  $\|r(x)\|_2^2 = \|Ax - b\|_2^2$ .

## 2. La Deducción: El Camino a las Ecuaciones Normales



- El Gradiente como Brújula

Definimos la función objetivo  $f(x) = \frac{1}{2} \|Ax - b\|_2^2$ . El punto mínimo ocurre cuando el gradiente es cero.

$$\nabla f(x) = A^T(Ax - b) = 0$$

- $\nabla f(x) = A^T(Ax - b) = 0$

Aplicando la regla de la cadena matricial, la condición de optimalidad nos conduce directamente al sistema de Ecuaciones Normales:  $A^T Ax = A^T b$ .

$$A^T Ax = A^T b$$

## 3. La Resolución: Descomposición de Cholesky

- Matriz de Covarianza  $A^T A$
- Factorización Única  $A^T A = G^T G$

$$A^T A = \begin{bmatrix} a_1 & a_2 & \cdots & a_n \\ a_1 & a_2 & \cdots & a_n \\ \vdots & \vdots & \ddots & \vdots \\ a_n & a_2 & \cdots & a_n \end{bmatrix}$$

Si la matriz  $A$  tiene rango completo  $n$ , entonces  $A^T A$  es una matriz Simétrica y Definida Positiva (SPD).

$$A^T A = \begin{bmatrix} \text{diagonal} \\ \text{triangular} \end{bmatrix} G^T \cdot \begin{bmatrix} \text{triangular} \\ \text{diagonal} \end{bmatrix} G$$

Al ser SPD, la matriz puede factorizarse mediante Cholesky en una matriz triangular superior  $G$  y su transpuesta, permitiendo resolver el sistema eficientemente mediante sustitución.

## 4. La Trampa: El Peligro del Mal Condicionamiento



- $\kappa_2(A^T A) = (\kappa_2(A))^2$

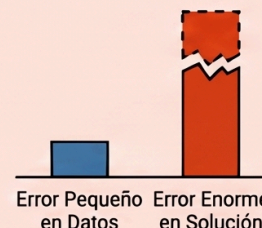
El "pecado original" de las Ecuaciones Normales es que el número de condición espectral se eleva al cuadrado, amplificando drásticamente la sensibilidad al error.

- Destrucción de la Precisión

En aritmética de doble precisión, este cuadrado del número de condición puede causar que los errores de redondeo devoren todas las cifras significativas de la solución.

- La Alternativa Estable: Descomposición QR

Aunque las Ecuaciones Normales son bellas analíticamente, numéricamente se desaconsejan; es preferible usar métodos como QR o SVD que mantienen el número de condición original  $\kappa_2(A)$ .



### Definición 4.2

Un conjunto de vectores  $\{x^1, \dots, x^p\}$  en  $\mathbb{R}^m$  se dice ortogonal si  $(x^i)^T x^j = 0$  para  $i \neq j$  y ortonormal si además vale  $(x^i)^T x^i = 1$ .

El complemento ortogonal de un subespacio  $\mathcal{K} \subseteq \mathbb{R}^m$  es

$$\mathcal{K}^\perp = \{y \in \mathbb{R}^m \mid y^T x = 0, \quad \forall x \in \mathcal{K}\}.$$

Una matriz  $Q \in \mathbb{R}^{m \times m}$  se dice ortogonal si  $Q^T = Q^{-1}$ , o sea  $Q^T Q = Q Q^T = I$ , por lo tanto  $Q = [q^1 \dots q^m]$  es ortogonal si y solo si sus columnas forman una base ortonormal de  $\mathbb{R}^m$ .

Note que si  $Q \in \mathbb{R}^{m \times n}$ , con  $m \geq n$ , tiene sus columnas ortonormales se obtiene que  $Q^T Q = I$  y por lo tanto  $(Qy)^T Qx = y^T x$  para todo  $x, y \in \mathbb{R}^n$ . Concluyendo que las matrices con columnas ortonormales preservan  $\|\cdot\|_2$ , i.e., para cualquier  $x \in \mathbb{R}^n$ ,

$$\|Qx\|_2 = \|x\|_2.$$



# Descomposición QR

Supongamos que queremos resolver el sistema lineal  $Ax = b$  y que  $A$  es una matriz de tamaño  $m \times n$  con  $m \geq n$ . Si multiplicamos el residuo por una matriz ortogonal  $Q$ , obtenemos:

$$\|Ax - b\|_2 = \|Q^T(Ax - b)\|_2 = \|Q^T Ax - Q^T b\|_2$$

Aplicaremos transformaciones ortogonales a la matriz  $A$  hasta hacerla triangular superior, de la misma forma que hicimos con Eliminación Gaussiana.

Veremos 2 maneras de hacerlo: **Rotaciones de Givens** y **Transformaciones de Householder**.

# Rotaciones de Givens

Las **Rotaciones de Givens** son transformaciones lineales ortogonales diseñadas para anular de forma precisa y selectiva un elemento individual de un vector o matriz.

Sea  $x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$ . Buscamos rotar por un ángulo  $\theta$  que anule la segunda coordenada:

$$y = \begin{bmatrix} r \\ 0 \end{bmatrix} = G \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

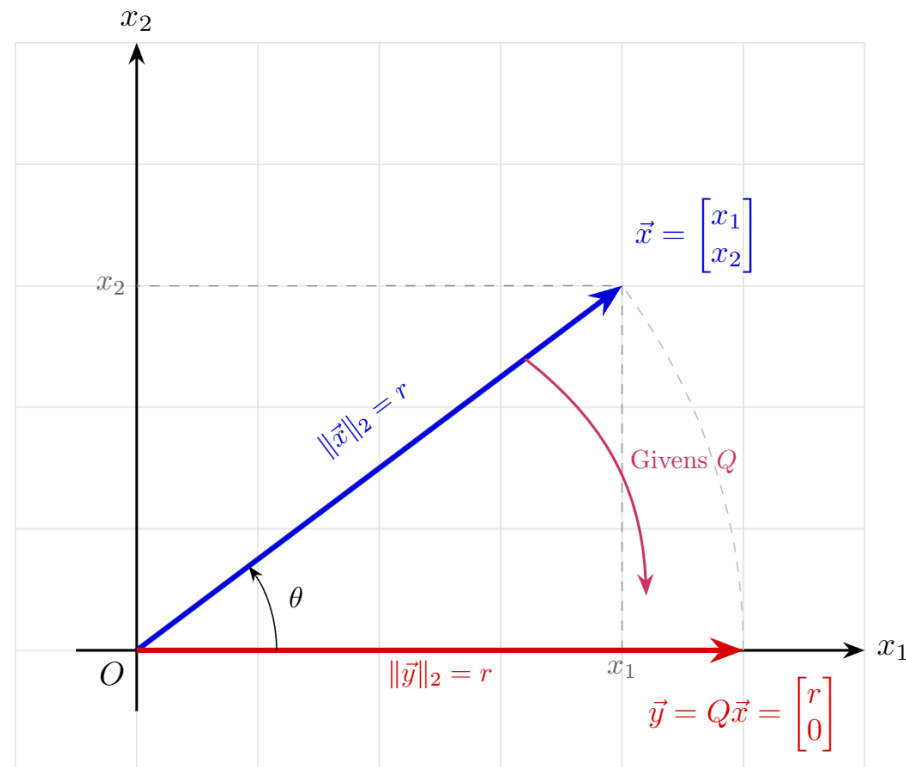
Al desarrollar el producto, la condición para anular el segundo componente es:

$$s \cdot x_1 + c \cdot x_2 = 0 \quad (\text{donde } c = \cos(\theta), \ s = \sin(\theta))$$

Sabiendo que  $c^2 + s^2 = 1$ , se resuelven los coeficientes trigonométricos como:

$$c = \frac{x_1}{\sqrt{x_1^2 + x_2^2}}, \quad s = \frac{-x_2}{\sqrt{x_1^2 + x_2^2}}$$

Esto produce una primera componente transformada:  $r = c \cdot x_1 - s \cdot x_2 = \|x\|_2$



## Generalización a $\mathbb{R}^m$

Para anular un elemento  $x_k \in \mathbb{R}$  utilizando el pivote  $x_i \in \mathbb{R}$  dentro de un vector de dimensión  $m$ , embebemos la rotación en la matriz identidad  $G_{ki} \in \mathbb{R}^{m \times m}$ :

$$G_{ki} = \begin{bmatrix} I & & & \\ & c & \dots & -s \\ & \vdots & I & \vdots \\ & s & \dots & c \\ & & & & I \end{bmatrix} \begin{matrix} \leftarrow \text{fila } i \\ \leftarrow \text{fila } k \end{matrix}$$

Cómo actúa esta transformación sobre todo el vector  $x$ ?

## Ejemplo

Consideremos el vector  $x = [3 \quad 4 \quad 12]^T \in \mathbb{R}^3$ . Queremos anular la tercera componente ( $x_3 = 12$ ) usando como pivote la primera ( $x_1 = 3$ ).

- **Coordenadas activas:**  $i = 1$  (fila pivote) y  $k = 3$  (fila a anular).
- **Cálculo de coeficientes  $c$  y  $s$ :**

$$c = \frac{3}{\sqrt{3^2 + 12^2}} = \frac{3}{\sqrt{153}}, \quad s = \frac{-12}{\sqrt{3^2 + 12^2}} = \frac{-12}{\sqrt{153}}$$



La matriz de rotación de Givens  $G_{31}$  se construye como:

$$G_{31} = \begin{bmatrix} c & 0 & -s \\ 0 & 1 & 0 \\ s & 0 & c \end{bmatrix}$$

Al multiplicar el vector  $x$  por nuestra matriz de rotación  $G_{31}$  obtenemos:

$$y = G_{31}x = \begin{bmatrix} c & 0 & -s \\ 0 & 1 & 0 \\ s & 0 & c \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} cx_1 - sx_3 \\ x_2 \\ sx_1 + cx_3 \end{bmatrix} = \begin{bmatrix} \sqrt{x_1^2 + x_3^2} \\ x_2 \\ 0 \end{bmatrix} = \begin{bmatrix} \sqrt{153} \\ 4 \\ 0 \end{bmatrix}$$

**RECORDAR:** En la práctica nunca vamos a construir la matriz de rotación explícitamente.

# Estabilidad Numérica

El cálculo de  $r = \sqrt{a^2 + b^2}$  puede causar *overflow* si  $a$  o  $b$  son muy grandes, o *underflow* y pérdida de precisión si son muy pequeños. Existe algo más robusto:

Si  $|a| \geq |b|$ :

1. Si  $b = 0$ , entonces  $c = 1$ ,  $s = 0$  y  $r = a$ .
2. Si  $b \neq 0$ , se calcula  $t = b/a$ . Como  $|a| \geq |b|$ , entonces  $|t| \leq 1$ .
3. Se calcula  $r = |a|\sqrt{1+t^2}$ . Esto evita elevar al cuadrado números muy grandes o muy pequeños.
4. Luego,  $c = \frac{a}{r} = \frac{a}{|a|\sqrt{1+t^2}} = \frac{\text{sign}(a)}{\sqrt{1+t^2}}$  y  $s = \frac{b}{r} = \frac{at}{|a|\sqrt{1+t^2}} = \frac{t \cdot \text{sign}(a)}{\sqrt{1+t^2}}$ .

Si  $|b| > |a|$  se hace un cálculo similar, pero con  $t = a/b$ .

# Algoritmo (Rotación de Givens)

Entradas:  $x_1, x_2 \in \mathbb{R}$ . Salidas:  $c, s \in \mathbb{R}$

- Si  $|x_1| + |x_2| = 0$ , retornar  $c = 1, s = 0$
- Si  $|x_2| > |x_1|$ :

$$t = -x_1/x_2$$

$$s = -\text{sign}(x_2)/\sqrt{1+t^2}, \quad c = t \cdot s$$

- Sino:

$$t = -x_2/x_1$$

$$c = \text{sign}(x_1)/\sqrt{1+t^2}, \quad s = t \cdot c$$

- Retornar  $c$  y  $s$ .

# Anulando Elementos

Para obtener la descomposición  $A = QR$  de una matriz  $A \in \mathbb{R}^{m \times n}$ , aplicamos transformaciones ortogonales en el lado izquierdo de  $A$  de manera secuencial para introducir ceros por debajo de la diagonal principal, trabajando columna por columna.

- Para la columna *activa*  $j$  (desde 1 hasta  $p = \min\{m - 1, n\}$ ), eliminamos cada elemento  $a_{kj}$  situado debajo de la diagonal (es decir, para  $k = j + 1, \dots, m$ ).
- Cada elemento  $a_{kj}$  se anula utilizando el elemento diagonal  $a_{jj}$  como pivote. Esto se logra multiplicando a la izquierda por la matriz de rotación embebida  $G_{kj}$ :

$$A \leftarrow G_{kj}A$$

# La Secuencia Global de Rotaciones

Reducimos la matriz de izquierda a derecha buscando la matriz triangular superior  $R$ :

- Para la primera columna ( $j = 1$ ):

$$A^{(1)} = G_{m1} \dots G_{31} G_{21} A \quad (m-1 \text{ rotaciones})$$

- Secuencia completa hasta la última columna ( $j = n$ , si  $m > n$ ):

$$R = (G_{m,n} \dots G_{n+1,n}) \dots (G_{m2} \dots G_{32})(G_{m1} \dots G_{21})A = Q^T A$$

Como  $G^T = G^{-1}$ , entonces  $Q \in \mathbb{R}^{m \times m}$  se reconstruye como:

$$Q = G_{21}^T G_{31}^T \dots G_{m1}^T G_{32}^T \dots G_{m,m-1}^T$$

# Impacto en las Filas de la Matriz

Cuando se aplica el operador  $G_{kj}$  a la izquierda de la matriz  $A$ , las propiedades de la transformación aseguran un comportamiento local óptimo.

1. Los elementos de las columnas  $1, \dots, j - 1$  que ya habían sido anulados permanecen en cero (0) porque la rotación actúa sobre componentes nulos.
2. Solo las filas  $j$  (pivote) y  $k$  (elemento a anular) sufren modificaciones en el rango de columnas restantes  $J = \{j, \dots, n\}$ :

$$\begin{bmatrix} A_{j,J} \\ A_{k,J} \end{bmatrix} \leftarrow \begin{bmatrix} c & -s \\ s & c \end{bmatrix} \begin{bmatrix} A_{j,J} \\ A_{k,J} \end{bmatrix}$$



# Algoritmo (Desc. QR por Rotaciones de Givens)

Entradas:  $A \in \mathbb{R}^{m \times n}$ . Salidas:  $Q \in \mathbb{R}^{m \times m}$ ,  $R \in \mathbb{R}^{m \times n}$

- Inicializar:  $Q = I_{m \times m}$
- Para  $j = 1 \dots p = \min\{m - 1, n\}$ : para  $i = j + 1 \dots m$ :
  - Si  $a_{i,j} \neq 0$ , definir  $\mathcal{I} = \{j, i\}$  y  $\mathcal{J} = \{j, \dots, n\}$ :  
Calcular:  $G = \begin{bmatrix} c & -s \\ s & c \end{bmatrix}$ ,  $c, s = \text{rotacion\_givens}(a_{j,j}, a_{i,j})$   
$$A_{\mathcal{I},\mathcal{J}} \leftarrow G A_{\mathcal{I},\mathcal{J}}$$
$$Q_{*,\mathcal{I}} \leftarrow Q_{*,\mathcal{I}} G^T$$
- Si  $m \leq n$  y  $a_{mm} = 0$ , definir  $\mathcal{J} = \{m, \dots, n\}$  y  
$$A_{m,\mathcal{J}} \leftarrow -A_{m,\mathcal{J}}, \quad Q_{*,m} \leftarrow -Q_{*,m}$$
- Retornar  $Q$  y  $R = A$

# Conteo Operacional

Para realizar una factorización QR en una matriz  $m \times n$ , debemos anular todos los elementos por debajo de la diagonal principal:

- Detalles en pizarra
- Aplicar una rotación de Givens a una matriz  $m \times n$  (es decir, actualizar dos filas de longitud  $n$ ) cuesta aproximadamente  $6n$  flops.
- Para triangularizar la matriz, necesitamos eliminar aproximadamente  $\frac{1}{2}n(2m - n - 1)$  elementos (para  $m \geq n$ ).
- Costo total (generar  $R$ ):  $\approx 3mn^2 - n^3$  flops. Para una matriz  $n \times n$ :  $\approx 2n^3$  flops.

## Teorema (Descomposición QR)

Sea  $A \in \mathbb{R}^{m \times n}$ . Entonces existe una matriz ortogonal  $Q \in \mathbb{R}^{m \times m}$  y una matriz triangular superior  $R \in \mathbb{R}^{n \times n}$  con  $r_{ii} > 0$  para todo  $i = 1 \dots n$ , tales que  $A = QR$ .

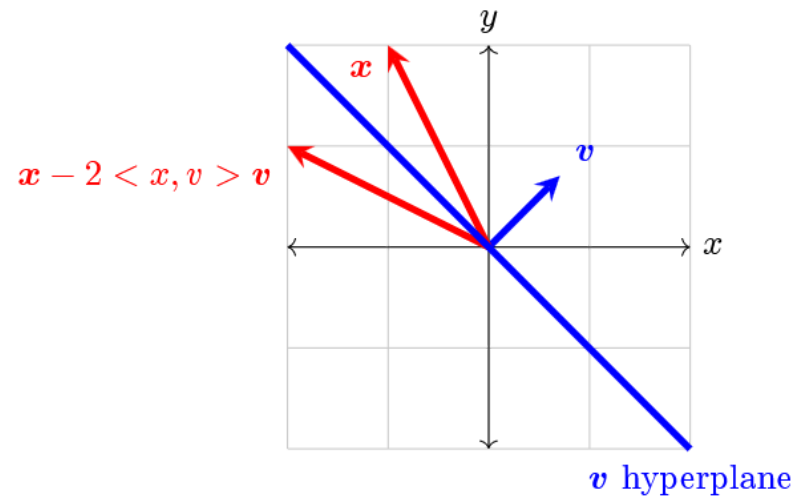
**Demostración:** ya lo hicimos! :D

## Hacia algo más eficiente

- Todo bien con tener una herramienta que nos permita obtener la descomposición  $QR$  de una matriz  $A \in \mathbb{R}^{m \times n}$ , pero podríamos desear una que sea más eficiente.
- Por ejemplo, las transformaciones de Gauss afectan directamente a toda la columna de  $A$  durante la descomposición LU, pero las rotaciones de Givens sólo afectan a dos filas a la vez.
- Vamos a aplicar esta misma idea para la próxima técnica.

# Transformaciones de Householder

- Una transformación de Householder (o reflexión de Householder) es una matriz que representa una reflexión respecto a un hiperplano. 
- Geométricamente, es una transformación lineal que refleja (o invierte) un espacio vectorial respecto a un plano o hiperplano.



# Propiedades de una Reflexión

Sea  $P \in \mathbb{R}^{m \times m}$  una *reflexión*, entonces cumple:

- Es **ortogonal**: una reflexión preserva la longitud de los vectores y los ángulos entre ellos, sólo que de forma espejada.
- Es **simétrica** ( $P^T = P$ ) e **involutiva** ( $P^2 = I$ ).
- Puede transformar cualquier vector dado  $x$  en otro vector  $y$ , siempre y cuando tengan la misma norma euclídea ( $\|x\|_2 = \|y\|_2$ ).

Existe este bicho o es una gallina esférica en el vacío?

# Proposición

Sea  $u \in \mathbb{R}^m \setminus \{0\}$ . Definimos

$$Q_u = I - \frac{2}{\|u\|_2^2} uu^T \in \mathbb{R}^{m \times m}$$

como *reflexión de Householder* respecto al hiperplano ortogonal al vector  $u$ . Entonces:

1.  $Qu = -u$ ,
2.  $Qv = v$  si  $u \perp v$ .
3.  $Q = Q^T$ ,
4.  $Q = Q^{-1}$ .

**Demostración en pizarra**

# Aplicación en QR

Para la primera columna  $x = a_1$ , buscamos una reflexión  $Q_1$  tal que  $Q_1x$  sea un múltiplo de  $e^1$ :

$$Q_1x = \begin{pmatrix} \sigma \\ 0 \\ \vdots \\ 0 \end{pmatrix} = \sigma e_1 \quad \text{con } |\sigma| = \|x\|_2$$

El vector normal al hiperplano de reflexión debe ser paralelo a la resta de  $x$  y su imagen reflejada  $\sigma e_1$ :

$$v = x - \sigma e_1$$



# Aplicación en QR

La matriz de Householder que realiza la reflexión sobre el hiperplano ortogonal a  $v$  es:

$$P_1 = I - \frac{2}{\|v\|_2^2} vv^T = I - \beta vv^T \quad \text{donde } \beta = \frac{2}{\|v\|_2^2}$$

El proceso se repite columna por columna de manera recursiva en subbloques cada vez más pequeños, y la matriz  $Q_k$  se define como:

$$Q_k = \begin{pmatrix} I_{k-1} & 0 \\ 0 & P_k \end{pmatrix}$$

Luego  $Q = Q_1 Q_2 \cdots Q_n$  y  $R = Q_n \cdots Q_2 Q_1 A$ .

# Estabilidad Numérica

- Para evitar errores numéricos, se puede pensar en la primera componente de  $u$  como:

$$u_1 = x_1 - \|x\|_2 = \frac{x_1^2 - \|x\|_2^2}{x_1 + \|x\|_2} = -\frac{\sum_{i=2}^m x_i^2}{x_1 + \|x\|_2}$$

- Luego podemos hacer que  $u_1 = 1$ , entonces:

$$u = \frac{x}{\gamma} = \begin{pmatrix} 1 \\ \frac{x_2}{\gamma} \\ \vdots \\ \frac{x_m}{\gamma} \end{pmatrix}, \quad \gamma = \begin{cases} x_1 - \|x\|_2 & \text{si } x_1 \leq 0 \\ -\frac{\sum_{i=2}^m x_i^2}{x_1 + \|x\|_2} & \text{si } x_1 > 0 \end{cases}$$

- Luego  $Q = I - \rho uu^T$  con  $\rho = 2\gamma^2/(\gamma^2 + \sum_{i=2}^m x_i^2)$ .

# Algoritmo (Reflexión de Householder)

Entrada:  $x \in \mathbb{R}^m$ . Salidas:  $u \in \mathbb{R}^m$ ,  $\rho \in \mathbb{R}$  tal que  $I - \rho uu^T x = \sigma e_1$

- Definir  $\sigma = \sum_{i=2}^m x_i^2$
- Si  $\sigma = 0$ , retornar  $u = 0$ ,  $\rho = 0$
- Definir  $\mu = \sqrt{\sigma + x_1^2}$
- Si  $x_1 \leq 0$ , definir  $\gamma = x_1 - \mu$ , sino definir  $\gamma = -\sigma/(x_1 + \mu)$
- Definir  $\rho = 2\gamma^2/(\sigma + \gamma^2)$
- Retornar  $u = [1, \quad x_2/\gamma, \quad \dots, \quad x_m/\gamma]^T$  y  $\rho$

# Algoritmo (Desc. QR por Refl. de Householder)

**Entrada:**  $A \in \mathbb{R}^{m \times n}$ . **Salidas:**  $Q \in \mathbb{R}^{m \times m}$  ortogonal,  $R \in \mathbb{R}^{m \times n}$  triangular superior

- Definir  $Q = I$ ,  $p = \min(m, n)$
- Para  $j = 1 \dots p$ , definir  $\mathcal{I} = \{j, \dots, m\}$ ,  $\mathcal{J} = \{j, \dots, n\}$ 
  - $u, \rho = \text{Householder}(A_{\mathcal{I},j})$
  - $w = \rho u$
  - $A_{\mathcal{I},\mathcal{J}} \leftarrow A_{\mathcal{I},\mathcal{J}} - w(u^T A_{\mathcal{I},\mathcal{J}})$
  - $Q_{\mathcal{I},*} \leftarrow Q_{*,\mathcal{I}} - (Q_{*,\mathcal{I}}w)u^T$
- Retornar:  $Q, R = A$

# Implementación Práctica

- Dónde se metió  $P$ ? 🙄
- Formar explícitamente la matriz de Householder requiere  $O(m^2)$  de memoria y  $O(m^2n)$  de operaciones inútiles.

- Explotamos que es una matriz de rango 1 para aplicarla directamente sobre  $A$ :

$$PA = (I - \beta vv^T)A = A - \beta v(v^T A)$$

- Este procedimiento reduce el costo a  $2mn$  **flops** por aplicación de reflector.

# Conteo Operacional (Crear R)

Sumamos el costo de las actualizaciones de rango 1 en cada paso  $k$  de dimensión decreciente  $m' = m - k + 1$  y  $n' = n - k + 1$ :

- Multiplicación vector-matriz ( $w^T = v^T A'$ ):  $\approx 2m'n'$  flops.
- Actualización de rango 1 ( $A' - \beta v w^T$ ):  $\approx 2m'n'$  flops.
- Costo total por paso  $k$ :  $\approx 4(m - k + 1)(n - k + 1)$  flops.

Sumando desde  $k = 1$  hasta  $n$ , obtenemos el costo  $2mn^2 - \frac{2}{3}n^3$  flops.

- Si  $m = n$ , el costo es de  $\frac{4}{3}n^3$  flops.
- Si  $m \gg n$ , el término  $2mn^2$  domina, con un costo aproximado de  $2mn^2$  flops.

## Algo para notar sobre $Q$

- En el algoritmo estamos construyendo  $Q$ , pero en realidad no necesitamos construirla.
- En su lugar, guardamos el vector de Householder  $u_k$  dentro de las posiciones que se acaban de anular por debajo de la diagonal (*pierdo información?*).
- El escalar  $\rho_k$  se guarda de forma paralela en un vector auxiliar de tamaño  $n$ .
- Si necesitamos multiplicar por  $Q^T$ , aplicamos la secuencia de actualizaciones:

$$Q^T b = (Q_n \cdots Q_2 Q_1) b$$

# Householder vs. Givens: ¿Cuándo usar cuál?

| Característica |  Householder ( <i>El Martillo</i> ) |  Givens ( <i>El Bisturí</i> ) |
|----------------|----------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| Acción         | Anula <b>toda</b> una subcolumna en un solo paso.                                                                    | Anula un <b>único elemento</b> a la vez (afecta 2 filas).                                                        |
| Geometría      | Reflexión respecto a un hiperplano.                                                                                  | Rotación plana en 2D.                                                                                            |
| Uso Ideal      | Matrices <b>densas</b> (más rápido y estable).                                                                       | Matrices <b>dispersas</b> , en banda o estructuradas.                                                            |



